Benchmarking training data generation strategies: Machine learning for molecular and materials property prediction

FREIE UNIVERSITÄT BERLIN

Minh Nghia Nguyen
Research Internship Presentation
Freie Universität Berlin - AG Paulus
07.07.2026

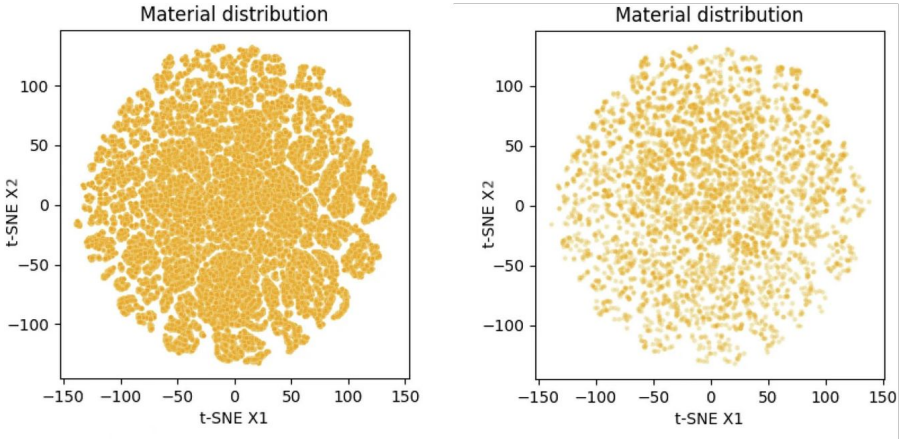
Table of Content

1. **Background & Motivation**
2. **Methods**
    - Machine Learning (ML) models
    - Active Learning (AL) strategies
3. **Results & Discussion**
    - Comparison Classical *vs.* Graph Neural Network models
    - Comparison between different AL strategies
4. **Conclusion**
5. **Outlook**

Background & Motivation

“More is better” illusion

- Resource-intensive data curation
- Highly redundant datasets
- Overestimated performance



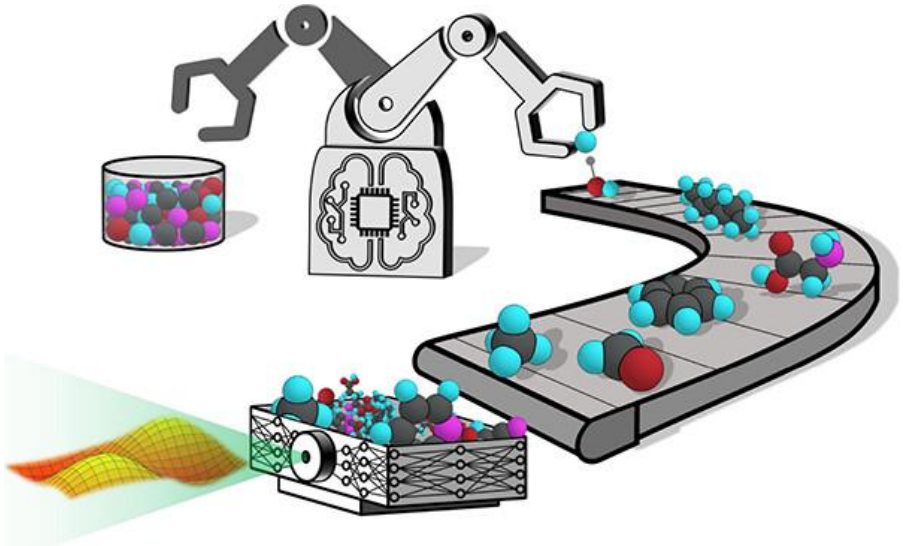
Distribution of whole and non-redundant dataset.

Q. Li, N. Fu, S. S. Omee, J. Hu, *npj Comput Mater* **10**, 245 (2024)
K. Li, D. Persaud, K. Choudhary, *et al.*, *Nat Commun* **14**, 7283 (2023)

Active Learning (AL)

Core idea:

- Start with small labeled set
- Train a model
- Ask: “Which unlabeled sample would teach the model the most?”
- Label only selected candidates



M. Kulichenko, B. Nebgen, N. Lubbers *et al.*, *Chemical Reviews* **124**, 13681-13714 (2024)
J. Bi, Y. Xu, F. Conrad, H. Wiemer, S. Ihlenfeldt, *Sci Rep* **15**, 37167 (2025)
A. S. Nair, L. Foppa, *Digital Discovery* **5**, 2366-2382 (2026)

Methods

Machine Learning (ML) models

Classical

Model: Committee (Ensemble)

- Random Forest (RF)
- Kernel Ridge + Nyström (KR)
- eXtreme Gradient Boosting (XGB)

Approach: Features are mapped using physical/chemical descriptors

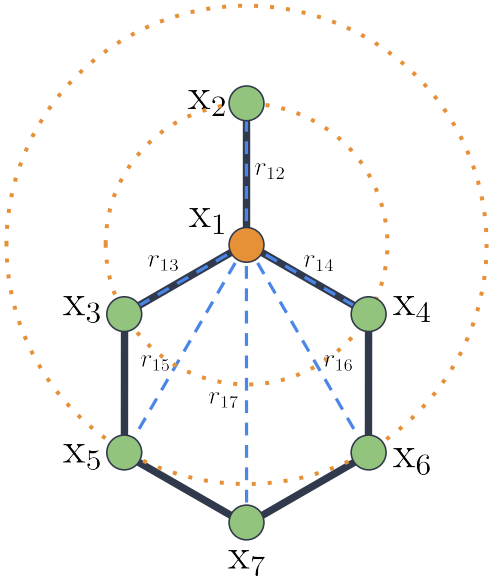
Graph Neural Network (GNN)

Model: SchNet

Approach: Direct mapping of 3D crystal/molecular structures

SchNet - Graph Convolutional Network

Graph representation



Convolution

$$x_i^{l+1} = (X^l * W^l)_i = \sum_j x_j^l \circ W^l(r_{ij})$$

x_i^l - current feature vector of atom *j*

W^l - continuous-filter function

r_{ij} - distance between atom *j* and *i*

$\circ$ - element-wise multiplication

K. T. Schütt , P.-J. Kindermans , H. E. Sauceda, *et al.*, *NeurIPS* **30**, (2017)

AL Strategies

Sampling methods:

- Random
  - Baseline
- Uncertainty (Query-by-Committee)
  - Empirical variance

$$\sigma^2(x) = \frac{1}{k}\sum_{\alpha}^{k}\left(\hat{y}^{\alpha}(x) - \bar{\hat{y}}(x)\right)^2$$

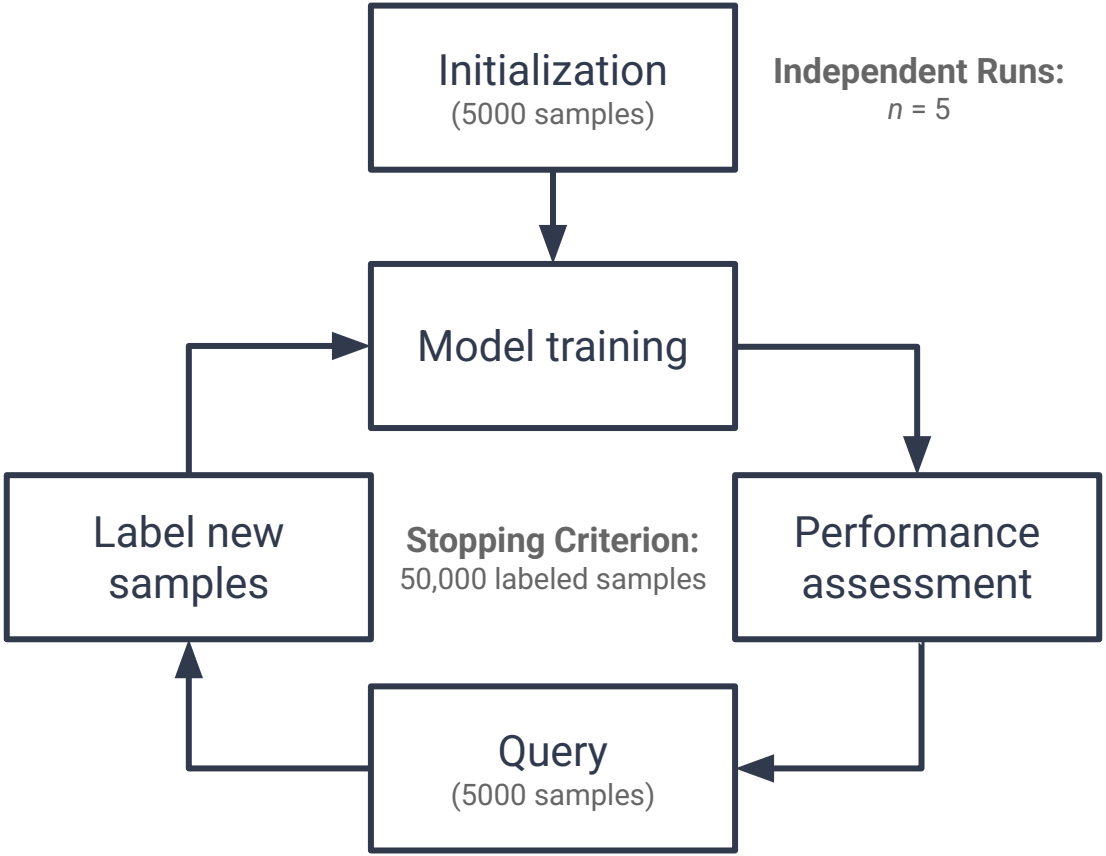
- Diversity
  - Typical clustering

Dataset splitting:

- Stratified splitting (target property)
  - Train - 80%
  - Test - 10%
  - Validation - 10%

R. Burbidge, J. J. Rowland, R. D. King, *In Intell. Data Eng. Autom. Learn.*, 209–218 (2007)
G. Hacohen, A. Dekel, and D. Weinshall, *In Int. Conf. Mach. Learn.*, 8175–8195 (2022)

AL Workflow

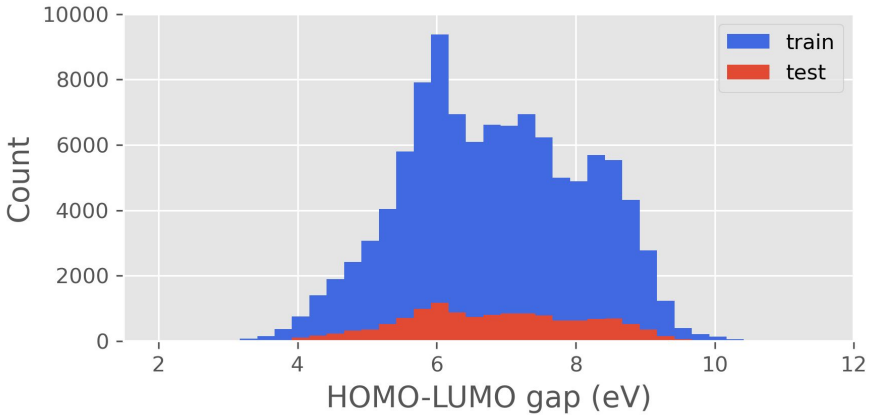

Initialization
(5000 samples)
Independent Runs:
n = 5
Model training
Label new samples
Stopping Criterion:
50,000 labeled samples
Performance assessment
Query
(5000 samples)

Results & Discussion

Molecular Dataset

QM9

- Size: 133,881 entries
- Type: small organic molecules
- Origin: B3LYP/6-31G
- Representation: 3D-structure
- Properties: 11 (ZPVE, HOMO-LUMO gap, Enthalpy, …)

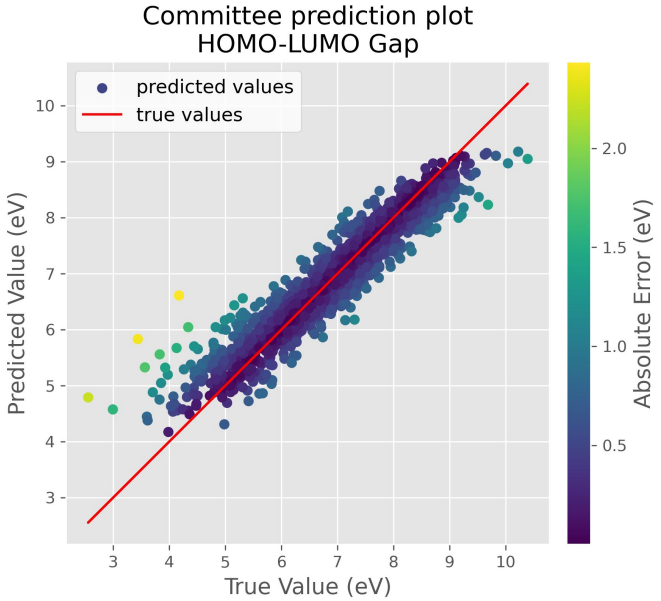


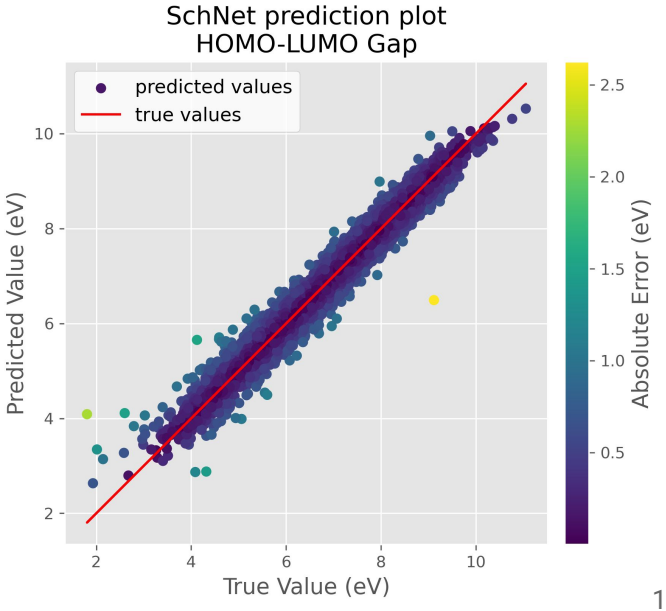
Train-test distribution of HOMO-LUMO gap in QM9 dataset.

L. Ruddigkeit, R. van Deursen, L. C. Blum, J.-L. Reymond, *J. Chem. Inf. Model*. **52**, 2864–2875 (2012)
R. Ramakrishnan, P. O. Dral, M. Rupp, O. A. von Lilienfeld, *Scientific Data* **1**, 140022 (2014)

QM9

Model:	Committee (RF + KR + XGB)	SchNet
Input:	features + fingerprints (Morgan, MACCS keys)	3D-structure
Target property:	HOMO-LUMO gap	HOMO-LUMO gap
Mean Average Error (MAE):	0.26 eV	0.16 eV
Root Mean Square Error (RMSE):	0.34 eV	0.21 eV





QM9 Classical *vs.* GNN

While both models achieve good results, SchNet reduced the prediction error by approximately 40% compared to classical approach

Table: Comparison of Classical *vs.* GNN models on QM9 dataset.

Type	Classical	GNN
Model	Committee (RF + KR + XGB)	SchNet
Training Size	107,106	107,106
MAE/RMSE	0.26/0.34 eV	0.16/0.21 eV
R^2	0.931	0.974
Training Time	<5 min	>4 h

QM9 + Classical | Active Learning

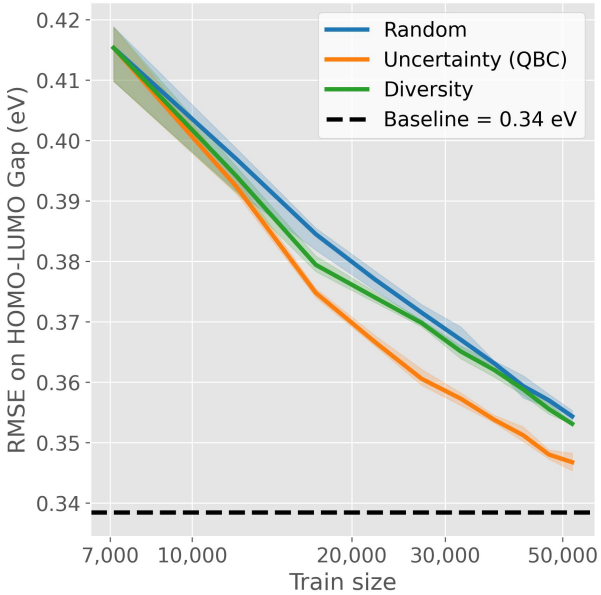
Model:

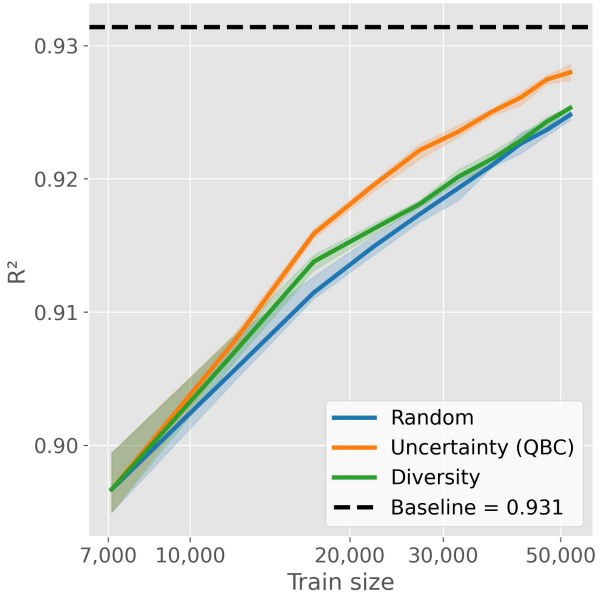
Committee (RF + KR + XGB)

Sampling strategies:

- Random
- Uncertainty (QBC)
- Diversity (Feature space)

Comparison of different sampling strategies on Committee model RMSE and R^2 AL curves.





QM9 + GNN | Active Learning

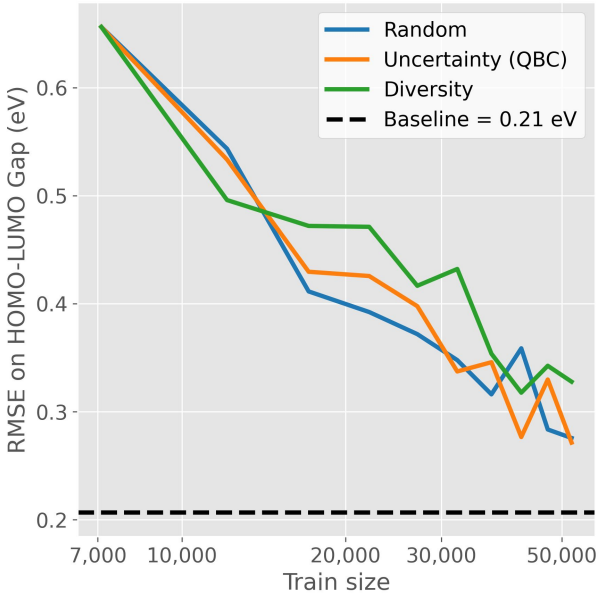
Model:

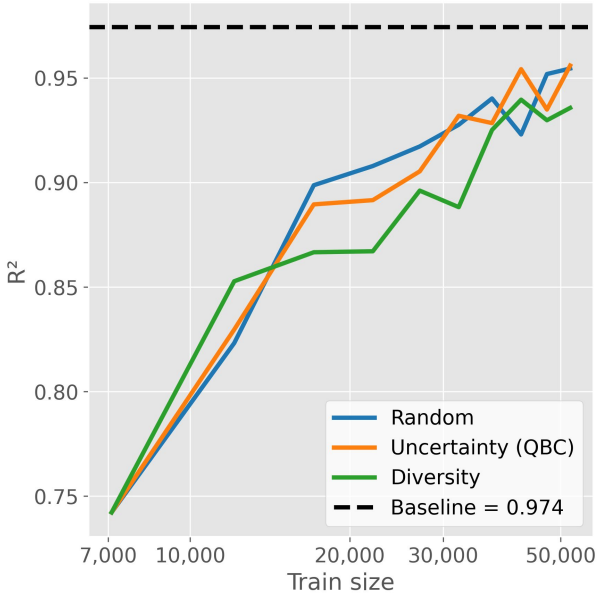
Schnet

Sampling strategies:

- Random
- Uncertainty (QBC)
- Diversity (Latent space)

Comparison of different sampling strategies on SchNet model RMSE and R^2 AL curves.





QM9 | AL Classical *vs.* GNN

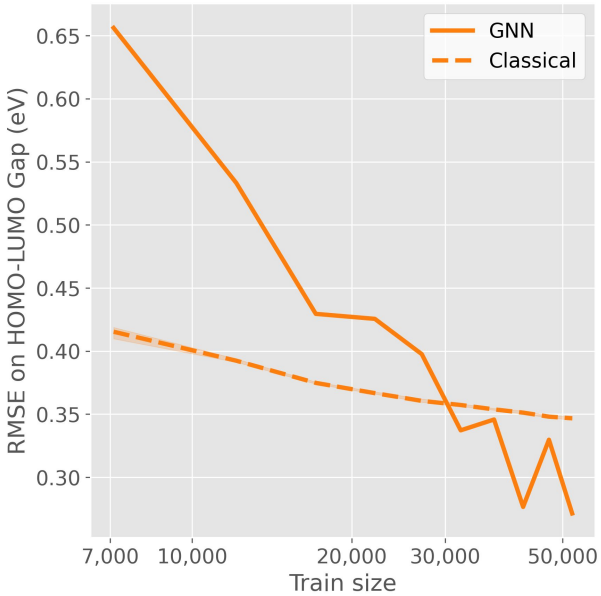
Models:

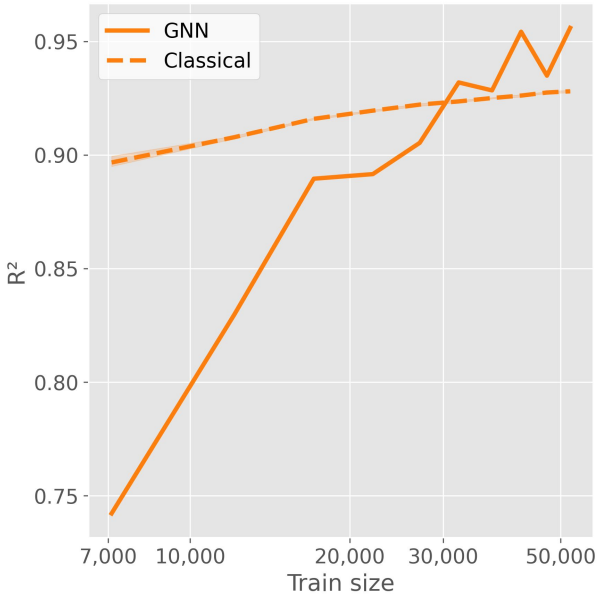
Committee *vs.* Schnet

Best sampling strategies:

- Uncertainty (QBC)

Comparison of Committee (Classical) and SchNet (GNN) models RMSE and R^2 AL curves.





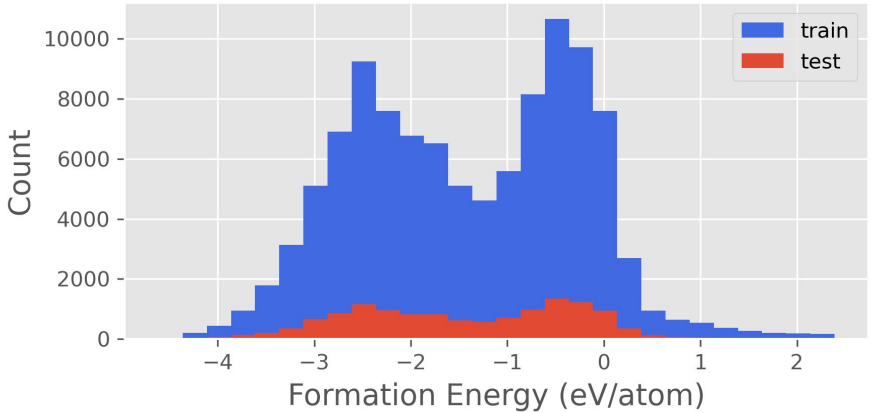
Materials Datasets

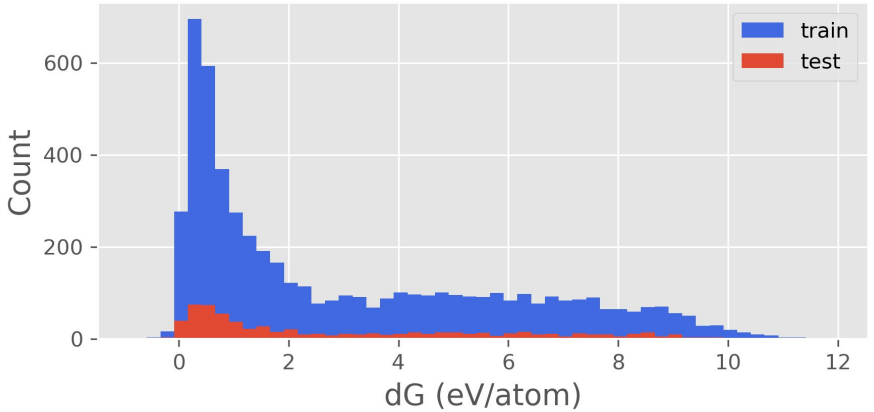
Matbench Materials Project

- Size: 132,752 entries
- Type: materials
- Origin: PBE
- Representation: 3D-structure

HSE06 dG

- Size: 6,870 entries
- Type: materials
- Origin: HSE06
- Representation: composition





Train-test distribution of Formation energy in Matbench MP and Pourbaix decomposition free energy in HSE06 dG datasets.

A. Jain, S.P. Ong, G. Hautier, W. Chen, et al., *APL Mater.* **1**, 011002 (2013)
A. Dunn, Q. Wang, A. Ganose, D. Dopp, A. Jain, *npj Comput Mater* **6**, 138 (2020)
A. S. Nair, L. Foppa, M. Scheffler, npj computational materials **11,** (2025)

HSE06 dG

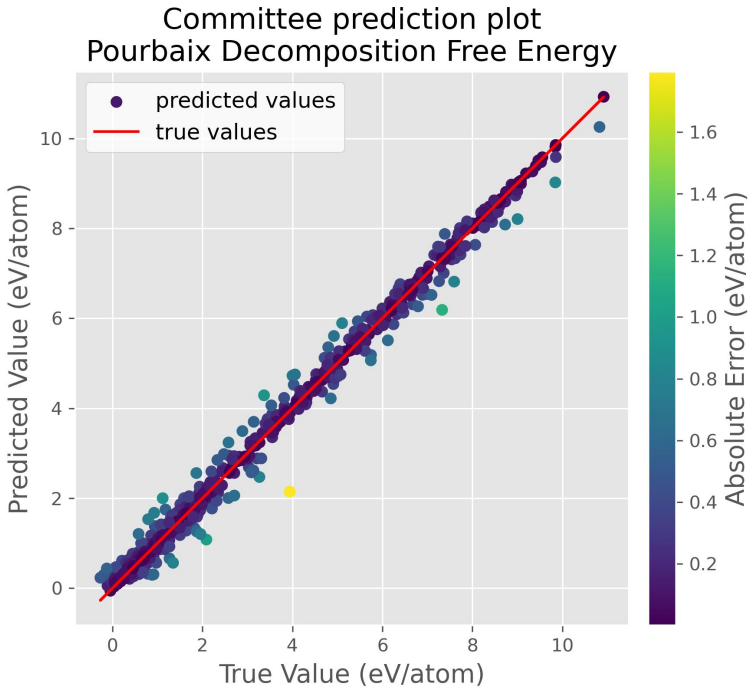
Model: Committee (RF + KR + XGB)

Input: composition features (Magpie)

Target property: Pourbaix Decomposition Free Energy

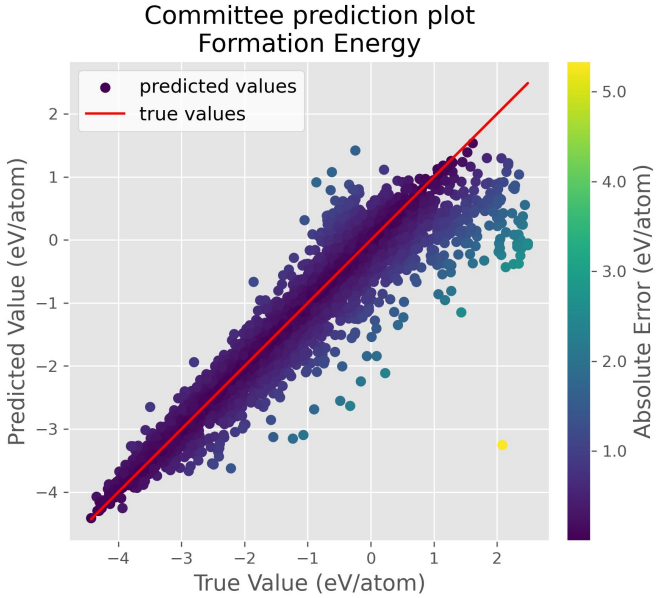
MAE: 0.26 eV

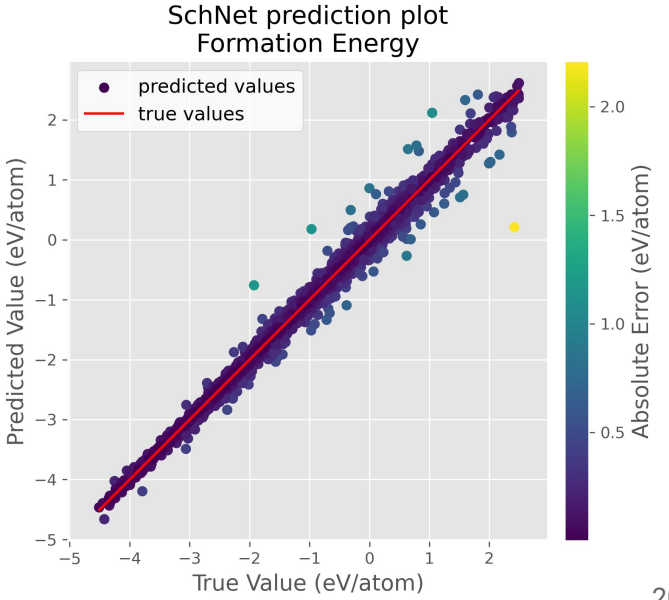
RMSE: 0.34 eV



Matbench Materials Project

Model:	Committee (RF + KR + XGB)	SchNet
Input:	composition features (Magpie)	3D-structure
Target property:	Formation Energy	Formation Energy
MAE:	0.14 eV	0.04 eV
RMSE:	0.27 eV	0.07 eV





Materials Classical *vs.* GNN

SchNet significantly outperforms classical models reducing prediction error by approximately 70%

Table: Comparison of Classical *vs.* GNN models on small and big materials datasets.

Type	Classical	Classical	GNN
Model	Committee (RF + KR + XGB)	Committee (RF + KR + XGB)	SchNet
Training Size	5,496	106,201	106,201
MAE/RMSE	0.16/0.24 eV/atom	0.14/0.27 eV/atom	0.04/0.07 eV
R^2	0.993	0.947	0.996
Training Time	<1 min	<5 min	>4 h

Matbench MP + Classical | Active Learning

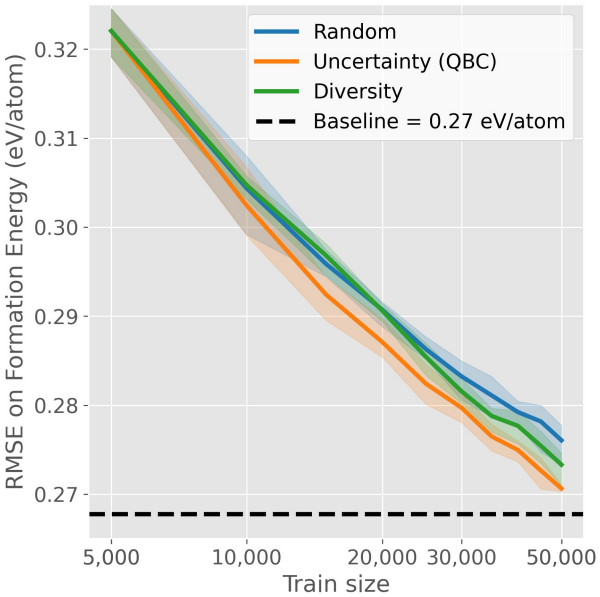
Model:

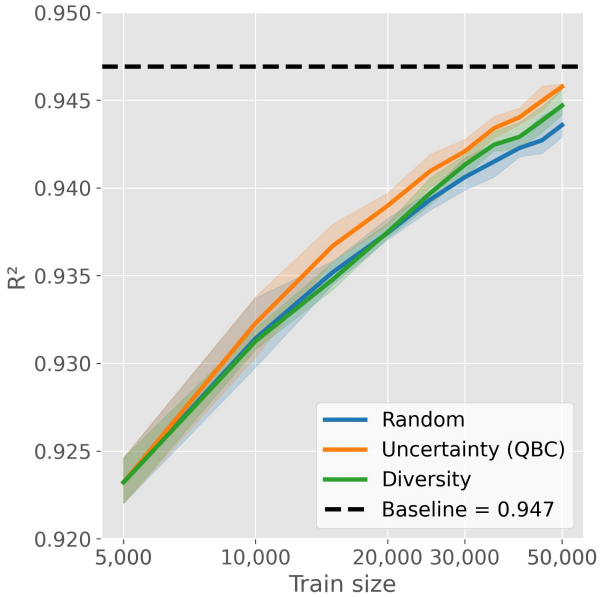
Committee (RF + KR + XGB)

Sampling strategies:

- Random
- Uncertainty (QBC)
- Diversity (Feature space)

Comparison of different sampling strategies on Committee model RMSE and R^2 AL curves.





Matbench MP + GNN | Active Learning

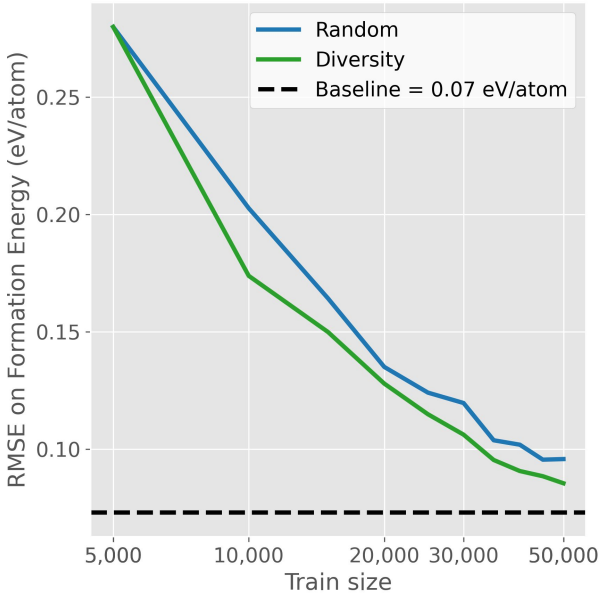
Model:

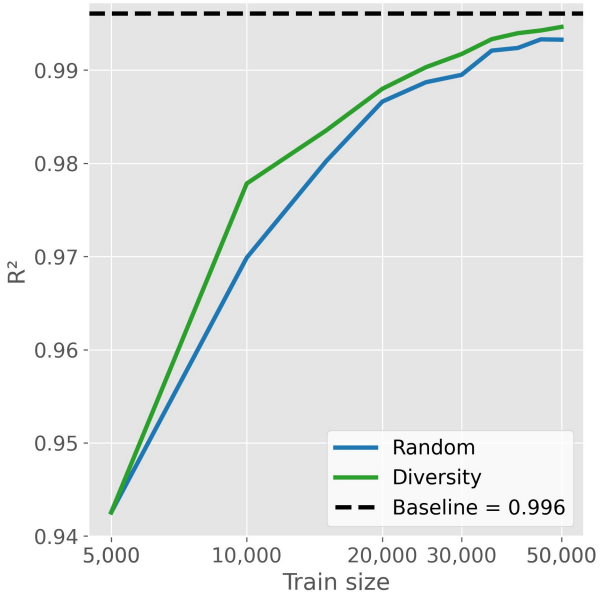
Schnet

Sampling strategies:

- Random
- Diversity (Latent space)

Comparison of different sampling strategies on SchNet model RMSE and R^2 AL curves.





Matbench MP | AL Classical *vs.* GNN

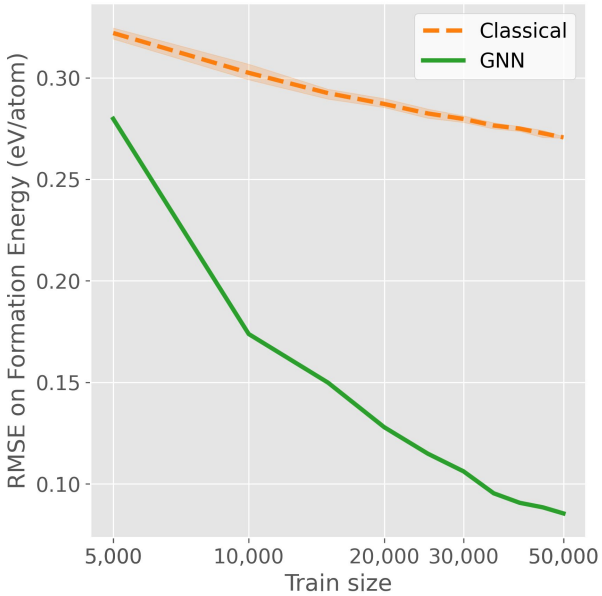
Models:

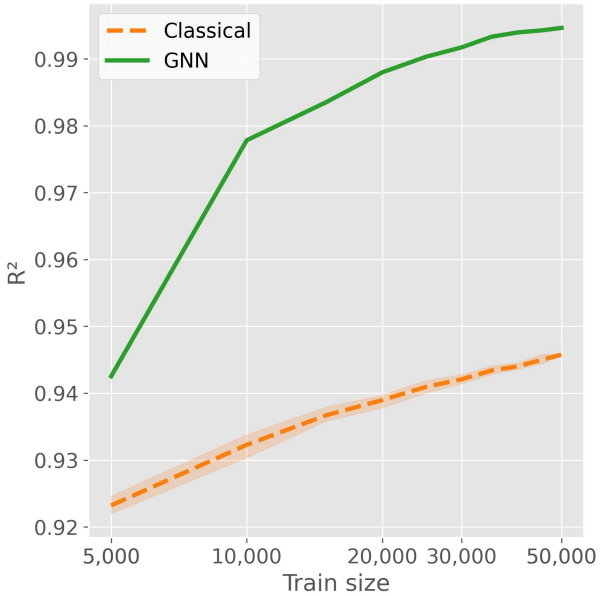
Committee *vs.* Schnet

Best sampling strategies:

- Uncertainty (QBC) - Classical
- Diversity (Latent space) - GNN

Comparison of Committee (Classical) and SchNet (GNN) models RMSE and R^2 AL curves.





Conclusion

1. **Optimal Architecture Selection**
   - Predictive performance scales with model complexity, establishing GNNs as the superior choice for high-accuracy property characterization at cost of computational resources.
2. **Data Efficiency via Active Learning**
   - Regardless of the model architecture, guided (uncertainty and diversity) sampling consistently outperforms random selection.
3. **The Discovery Pipeline Trade-off**
   - The integration of GNNs and uncertainty-based sampling creates a strategic "accuracy-cost" balance, enabling high-fidelity results while significantly minimizing the total volume of labeled data required.

Future Directions and Outlook

- Extend the current framework to predict multiple physical-chemical properties simultaneously.
- Implement more efficient sampling strategies.
- Implement hybrid sampling strategies, utilizing several methods simultaneously.
- Extend the current framework for Machine-learned interatomic potentials (MLIPs)
- Explore dataset pruning
- Explore more sophisticated GNN models

Appendix

Uncertainty (QBC)

$$\sigma^2(x) = \frac{1}{k} \sum_{\alpha}^{k} \left(\hat{y}^{\alpha}(x) - \bar{\hat{y}}(x) \right)^2$$

$\sigma^2(x) = \text{variance}$
$k = \text{number of committee members}$
$\hat{y}^{\alpha}(x) = \text{prediction at input } x \text{ made by committee member } \alpha$
$\bar{\hat{y}}(x) = \text{average committee prediction}$

R. Burbidge, J. J. Rowland, R. D. King, *In Intell. Data Eng. Autom. Learn.*, 209–218 (2007)

Typical Clustering

$$\text{Typicality}(z) = \left(\frac{1}{K} \sum_{z_i \in K\text{-NN}(z)} \|z - z_i\|_2 \right)^{-1}$$

$$x_j^* = \arg\max_{x \in C_j} \left(\text{Typicality}(x) \right)$$

Queried batch:

$$Q_t = \{x_j^* : C_j \in B\}$$

$C_j = \text{uncovered cluster}$
$B = \text{query budget}$

G. Hacohen, A. Dekel, and D. Weinshall, *In Int. Conf. Mach. Learn.*, 8175–8195 (2022)

Adam optimizer

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

D. P. Kingma, J. Ba, *ICLR* **3**, (2015)

SchNet hyperparameters

QM9

```
model = NeuralNetRegressor(
    module=SchNetRegressorModule,
    module__hidden_channels=64,
    module__num_filters=256,
    module__num_interactions=6,
    module__num_gaussians=50,
    module__cutoff=6.0,
    criterion=nn.MSELoss,
    optimizer=torch.optim.Adam,
    lr=1e-4,
    batch_size=32,
    max_epochs=150,
    iterator_train=DataLoader,
    iterator_valid=DataLoader,
    iterator_train__shuffle=True,
    iterator_train__num_workers=0,
    iterator_valid__num_workers=0,
    train_split=predefined_split(InferenceDataset(val_dataset, target_idx=target_idx)),
    device=device,
    verbose=1,
    callbacks=[
        ('lr_scheduler', LRScheduler(
            policy=ReduceLROnPlateau,
            monitor='valid_loss',
            mode='min',
            factor=0.5,
            patience=5,
            min_lr=1e-6,
        )),
        ('early_stopping', EarlyStopping(
            monitor='valid_loss',
            patience=20,
            lower_is_better=True,
            threshold=0.001,
            threshold_mode='rel',
        )),
        ('checkpoint', Checkpoint(
            f_params=None,
            f_history=None,
            f_criterion=None,
            f_optimizer=None,
            monitor='valid_loss_best',
            load_best=True,
        )),
    ],
)
```

Matbench MP

```
model = NeuralNetRegressor(
    module=SchNetRegressorModule,
    module__hidden_channels=64,
    module__num_filters=128,
    module__num_interactions=6,
    module__num_gaussians=50,
    module__cutoff=5.0,
    module__readout='mean',
    criterion=nn.MSELoss,
    optimizer=torch.optim.AdamW,
    optimizer__weight_decay=1e-6,
    lr=1e-4,
    batch_size=32,
    max_epochs=150,
    iterator_train=DataLoader,
    iterator_valid=DataLoader,
    iterator_train__shuffle=True,
    iterator_train__num_workers=0,
    iterator_valid__num_workers=0,
    train_split=predefined_split(InferenceDataset(val_dataset, target_idx=target_idx)),
    device=device,
    verbose=1,
    callbacks=[
        ('lr_scheduler', LRScheduler(
            policy=ReduceLROnPlateau,
            monitor='valid_loss',
            mode='min',
            factor=0.5,
            patience=5,
            min_lr=1e-6,
        )),
        ('early_stopping', EarlyStopping(
            monitor='valid_loss',
            patience=20,
            lower_is_better=True,
            threshold=0.001,
            threshold_mode='rel',
        )),
        ('checkpoint', Checkpoint(
            f_params=None,
            f_history=None,
            f_criterion=None,
            f_optimizer=None,
            monitor='valid_loss_best',
            load_best=True,
        )),
```

Classical models' hyperparameters

Molecules datasets

```python
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(
    n_estimators=100,
    max_features='sqrt',
    min_samples_leaf=2,
    n_jobs=-1,
    random_state=42,
)
# rf.fit(X_train, y_train)
# joblib.dump(rf, 'models/rf/qm9_small/rf_gap_qm9_s
```

```python
from sklearn.kernel_ridge import KernelRidge
from sklearn.kernel_approximation import Nystroem #
from sklearn.linear_model import Ridge
kr = Pipeline([
    ('nystroem', Nystroem(
        kernel='laplacian',
        n_components=2500,
        gamma=0.01,          # This is the most imp
        n_jobs=-1,
        random_state=42,
    )),
    ('ridge', Ridge(alpha=0.01))
])

# kr.fit(X_train,y_train)
# joblib.dump(kr, 'models/kr/qm9_small/kr_gap_qm9_s
```

```python
from xgboost import XGBRegressor
xgb = XGBRegressor(
    booster='gbtree',
    colsample_bytree=0.6,
    learning_rate=0.1,
    max_depth=4,
    min_child_weight=3,
    n_estimators=100,
    reg_alpha=0.1,
    reg_lambda=1.0,
    subsample=0.8,
    n_job=-1,
)
# xgb.fit(X_train,y_train)
# joblib.dump(xgb, 'models/xgb/qm9_small/xgb_gap_qm
```

Materials dat

```python
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(
    n_estimators=500,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features=0.8,
    n_jobs=-1,
    random_state=42
)
# rf.fit(X_train,y_train)
# joblib.dump(rf, 'models/rf/matbench/rf_eform_matb
```

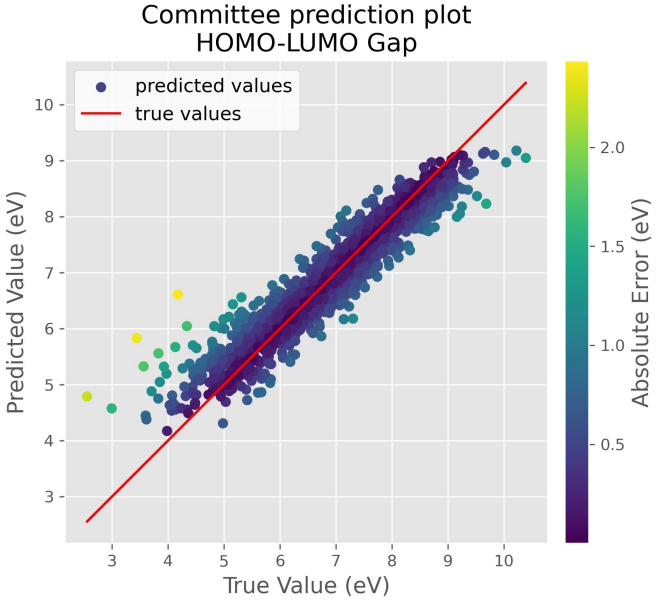
```python
from sklearn.kernel_approximation import Nystroem #
from sklearn.linear_model import Ridge
kr = Pipeline([
    ('nystroem', Nystroem(
        kernel='laplacian',
        n_components=2500,
        gamma=0.01,          # This is the most imp
        n_jobs=-1,
        random_state=42,
    )),
    ('ridge', Ridge(alpha=0.01))
])

# kr.fit(X_train,y_train)
# joblib.dump(kr, 'models/kr/matbench/kr_eform_matb
```

```python
from xgboost import XGBRegressor
xgb = XGBRegressor(
    booster='gbtree',
    colsample_bytree=0.8,
    learning_rate=0.1,
    max_depth=4,
    min_child_weight=3,
    n_estimators=500,
    reg_alpha=1.0,
    reg_lambda=1.0,
    subsample=0.6,
)
# xgb.fit(X_train,y_train)
# joblib.dump(xgb, 'models/xgb/matbench/xgb_eform_m
```

QM9

Model:	Committee (RF + KR + XGB)
Input:	features + fingerprints (Morgan, MACCS keys)
Target property:	HOMO-LUMO gap
Mean Average Error (MAE):	0.26 eV
Root Mean Square Error (RMSE):	0.34 eV



Model:	SchNet
Input:	3D-structure
Target property:	HOMO-LUMO gap
MAE:	0.16 eV
RMSE:	0.21 eV

